

Here's the full harness design as planned, in reference-table form.

System Layers Overview

Layer	Component	Purpose
1. Ingestion	File router, page classifier, AnyDoc, Docling	PDF/MD → clean per-page text
2. Chunking	Structure-aware splitter, contextual preamble LLM	Text → semantically coherent, situated chunks
3. Storage	Qdrant, metadata schema	Chunks → searchable vector index
4. Retrieval	Dense + sparse + OKF, RRF, dedup, rerank	Query → ranked relevant chunks
5. Generation	Context assembly, LLM answer	Chunks + query → grounded answer
6. Caching	Query / embedding / retrieval cache	Reduce repeat cost
7. Agent/Harness	ReAct loop, tools, loop control	Orchestrates search → reason → answer
8. Verification	Citation validator, NLI groundedness	Answer → checked/repaired answer
9. State	Registry, counters, history	Tracks everything the loop needs

1. Ingestion Layer

Decision	Choice	Why
Input formats	PDF, MD only	Scoped for now
Routing unit	Per-page (PDF), whole-file (MD)	MD has no scan problem; PDF pages vary
Page classifier signals	Text char count, image coverage ratio, table detection (all via PyMuPDF, structural not ML)	Fast, deterministic, no rendering needed
Clean text page	→ AnyDoc/PyMuPDF native extraction	Fast, no OCR risk
Scanned OR mixed (table+scan+text)	→ Docling (whole page)	Docling has OCR + table recognition; simpler than region-splitting

Decision	Choice	Why
Docling batching	Once per document , sliced by page via <code>item.prov[].page_no</code>	Avoids reloading OCR models per page (was the slowness bug)
Post-extraction check	<code>ExtractionQualityChecker</code> — short/garbled/junk-ratio	Catches garbled-font and bad-pre-existing-OCR edge cases
Routing audit	<code>IngestionRoutingLog</code>	Every decision logged with reason — debuggable, not silent

2. Chunking Layer

Decision	Choice	Why
Splitting method	<code>RecursiveCharacterTextSplitter</code> + <code>MarkdownHeaderTextSplitter</code>	Respects headers/sentence boundaries, not raw char slicing
Contextual preamble	LLM call per chunk, 1-2 sentences	Fixes orphan-chunk / pronoun-ambiguity problem
Context window	Whole-doc if under threshold, else windowed (prev+next chunk)	Cost control on long docs
Preamble LLM	Groq: <code>qwen/qwen3.8-27b</code> primary, <code>openai/gpt-oss-20b</code> fallback	Cheap task, non-reasoning model avoids wasted "thinking" tokens
Fallback trigger	2 retries on primary → switch model	Rate-limit resilience
Concurrency	<code>ThreadPoolExecutor</code> , workers sized to RPM limit	I/O-bound calls, order preserved via indexed futures
Separation of concerns	Preamble = semantic context; metadata = structured fields	Don't bury doc/page info in LLM text

3. Storage Layer

Decision	Choice	Why
Vector DB	Qdrant (local embedded path for Colab, server URL for prod)	Payload filtering + vector search together

Decision	Choice	Why
Metadata fields	<code>doc_id</code> , <code>doc_version</code> , <code>source_type</code> , <code>page_num</code> , <code>section_title</code> , <code>ingestion_route</code>	Needed for prefiltering, citation, audit
Embedding target	<code>contextual_text</code> (preamble + raw), not <code>raw_text</code> alone	Preamble improves semantic match
Batch upsert	Yes	One round trip vs. one per chunk

4. Retrieval Layer

Decision	Choice	Why
Dense	Qdrant vector search	Semantic match
Sparse	BM25 (in-memory; swap for Elasticsearch in prod)	Keyword/exact-term match (clause numbers, etc.)
OKF	Secondary external knowledge source	Enrichment, not equal-vote
Fusion	Two-stage RRF : (dense+sparse) fused first, then fused-with-OKF	OKF shouldn't outvote internal corpus consensus (fintech-specific design call)
Dedup	Embedding-similarity threshold, run before 2nd RRF stage	Avoid wasting rerank slots on near-duplicates
Rerank	Cross-encoder (<code>ms-marco-MiniLM-L-6-v2</code>)	Precision pass on final candidates

5. Generation Layer

Decision	Choice	Why
Prompt structure	System → Context → Question (fixed order)	Model reads everything before answering
Context formatting	<code>[chunk_id]</code> Document / Page / Content per chunk	Enables real citation
Relevance fallback	Below <code>MIN_RELEVANCE_SCORE</code> → "insufficient information"	Primary hallucination guardrail

Decision	Choice	Why
Claim extraction	XGrammar structured output (not regex)	Reliable regardless of prose style; regex kept only as degrade fallback

6. Caching Layer

Tier	What it stores	Skip condition
Query cache	Full answer, keyed by query+doc_version	Exact match only
Embedding cache	Query vector	Same query text
Retrieval cache	Candidate chunks	Semantically similar queries
Invalidation	By <code>doc_version</code> hash on re-ingestion	Avoid serving stale-policy answers

7. Agent / Harness Layer (ReAct Loop, LangGraph)

Tools:

Tool	Purpose
<code>list_collections</code> / <code>list_documents</code>	Discovery
<code>search_documents(query, filters?)</code>	Primary hybrid search
<code>search_documents_reformulated(query, reason)</code>	Forces articulated retry, not blind reword
<code>get_context(chunk_id, window)</code>	Expand around a hit
<code>get_page(doc_id, page_num)</code>	Direct jump
<code>get_table_of_contents(doc_id)</code>	Cheap structural lookup
<code>compare_chunks(a, b)</code>	Multi-hop comparison
<code>check_document_freshness(doc_id)</code>	Version check
<code>think</code>	Scratchpad

Loop control:

Cap	Purpose
<code>max_iterations</code>	Overall reasoning budget

Cap	Purpose
<code>max_token_budget</code>	Cost control
<code>max_search_attempts</code>	Separate from iterations — limits reworded-query spam
Stuck-loop detector	Registry size unchanged over N searches → forced degrade
Degrade behavior	Best-grounded partial answer + explicit caveat, never silent fail or bluff

8. Verification Layer

Step	Order	Behavior
Citation validator	1st (code-only, cheapest)	Hard fail if <code>chunk_id</code> not in registry — hallucination, not judgment call
Groundedness check	2nd (only if citations valid)	NLI entailment model (<code>cross-encoder/nli-deberta-v3-base</code>), run on all claims per your instruction
Retry	Up to <code>MAX_VALIDATION_RETRIES</code>	Specific failure reason fed back, not generic "try again"
Failure routing	Over-claim → regenerate (inner loop); missing coverage → re-retrieve (outer loop)	Different root causes need different fixes
Final fallback	Strip ungrounded claims, keep the rest	Don't zero out a partially-good answer

9. State Layer

Field	Purpose
<code>retrieved_chunk_registry</code>	Ground truth for citation validator
<code>search_attempts</code> / <code>registry_size_history</code>	Stuck-loop detection
<code>token_budget_used</code>	Budget enforcement
<code>validation_retries</code>	Verification loop control
Doc-version awareness	Prevents citing stale re-ingested docs

Cross-Cutting Design Principle

The LLM only makes judgment calls (what to search, when to stop, what a claim means). Everything checkable — citation existence, budget limits, retry counts, cache validity, document freshness, extraction quality — is enforced in code, never trusted to model self-report.

Where we actually are right now: Ingestion  (tested), Chunking  (tested, concurrent), Storage — wiring in progress (local Qdrant fix + batch upsert + embedding generator just added, awaiting your test output from Cells 13–14). Retrieval, Generation, Agent/Harness, Verification are all coded from earlier but **not yet tested against real stored data** — that's the natural next step once storage is confirmed working.